Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Computación

Carrera:

Software

Asignatura:

Aplicaciones Distribuidas

Actividad:

TA-IND-01 · Informe Académico Individual

Análisis crítico de sistemas distribuidos: Google Spanner y Amazon DynamoDB

Estudiante:

Bryan Humberto Lombeida Escaleras

Paralelo:

«A»

Fecha:

Viernes 13 de noviembre de 2026

Enlace del repositorio de GitHub

https://github.com/ameraj2-bit/ta-ind-01-sistemas-distribuidos.git

Resumen

Este informe analiza Google Spanner, una base de datos distribuida que permite realizar transacciones sobre datos repartidos entre varias zonas o regiones. A partir de documentación técnica y estudios publicados, se examinan su arquitectura, la comunicación entre componentes, la tolerancia a fallos y las decisiones de consistencia, disponibilidad y latencia. La comparación con las tablas globales de Amazon DynamoDB en modo de consistencia eventual multirregional permite identificar qué garantías convienen según las necesidades de la aplicación. En el emulador oficial de Spanner se realizaron dos pruebas. La cancelación de una transferencia conservó ambos saldos y la confirmación aplicó el débito y el crédito. Al detener el contenedor, la consulta quedó sin respuesta dentro del límite de quince segundos. Tras reiniciarlo, fue necesario crear de nuevo la instancia y cargar los datos iniciales. El análisis indica que Spanner resulta adecuado cuando los cambios de una transacción deben observarse juntos. DynamoDB en el modo estudiado puede ajustarse mejor cuando se prioriza confirmar escrituras en una región y se aceptan lecturas temporalmente desactualizadas en otras.

Palabras clave: Spanner, DynamoDB, replicación, consistencia, tolerancia a fallos.

1. Introducción

Google Spanner aborda el problema de mantener datos coherentes cuando una aplicación crece y necesita repartirlos entre varias zonas o regiones. Por ejemplo, si una operación descuenta inventario y registra una reserva, ambos cambios deben confirmarse juntos. Spanner combina transacciones atómicas con replicación síncrona para ofrecer esas garantías sobre datos distribuidos [1, 2].

Google utiliza Spanner en servicios como Gmail, YouTube y Google Fotos. Entre los clientes que Google Cloud menciona están Shopify y Mercado Libre. En enero de 2026, Google informó que Spanner manejaba más de 17 exabytes de datos y superaba los 6 000 millones de consultas por segundo en picos de actividad. Estas cifras describen la escala que Google reportó para el sistema en conjunto [3].

Elegí Spanner porque su diseño permite estudiar una decisión concreta de los sistemas distribuidos: qué gana una aplicación al coordinar sus escrituras y qué ocurre si las réplicas dejan de comunicarse. La comparación con DynamoDB permite examinar esa decisión ante un modelo de replicación distinto. El emulador oficial también me permitió observar desde el cliente dos situaciones: la confirmación o cancelación de una transferencia y la interrupción de una consulta al detener el contenedor.

2. Arquitectura del sistema

2.1. Componentes y funciones

La aplicación se comunica con Spanner mediante una API o una biblioteca cliente. En una configuración regional base, cada rango de datos, o *split*, tiene tres réplicas con derecho a voto distribuidas en zonas distintas de la misma región. Una de ellas actúa como líder y coordina las escrituras de ese rango [2, 4].

2.2. Comunicación entre componentes

El servicio encamina la solicitud a la réplica líder. El líder registra el cambio, lo envía en paralelo a las otras réplicas con voto y espera el acuerdo de una mayoría antes de confirmar la escritura al cliente. En la configuración representada, el quórum es de dos votos de tres, incluido el del líder. Si una réplica queda fuera de servicio, las otras dos pueden seguir confirmando escrituras siempre que se comuniquen y cuenten con un líder [2].

2.3. Diagrama original y evolución desde el esquema a mano

El boceto del Anexo A identifica la aplicación, el servicio, la réplica líder y dos réplicas adicionales. En la versión digital, el rango de datos deja de aparecer como un bloque separado y pasa a delimitar el grupo de réplicas que guardan sus copias. El diagrama también ubica las tres zonas dentro de una sola región y numera el recorrido desde la solicitud inicial hasta la respuesta al cliente. Las flechas azules muestran el envío de solicitudes y escrituras, las moradas muestran los votos y las verdes indican la confirmación.

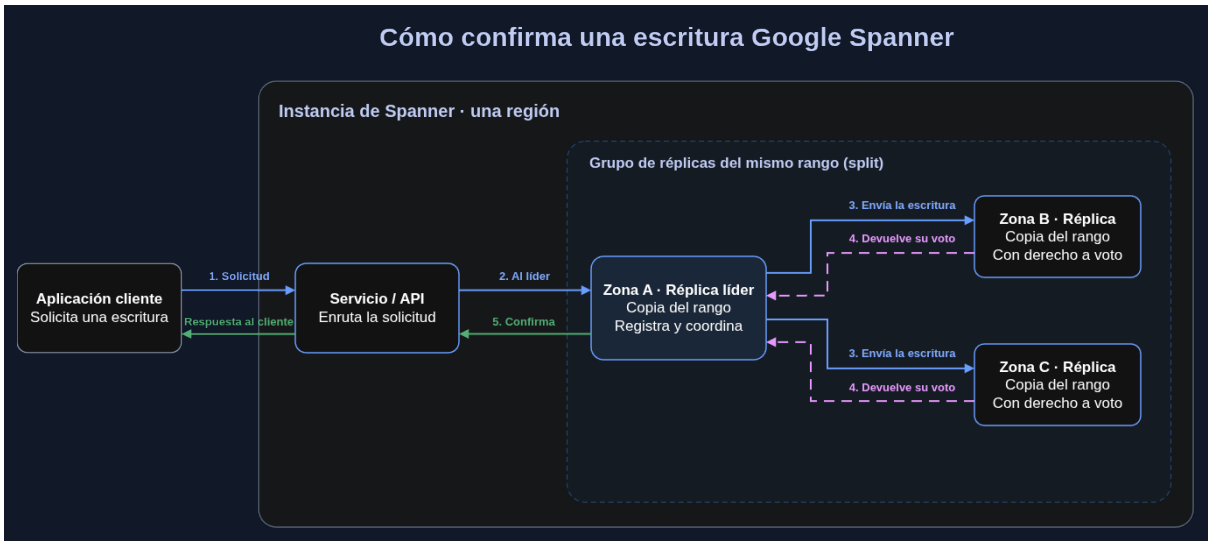


Figura 1: Recorrido de una escritura en Spanner. Diagrama elaborado a partir del boceto del Anexo A y de la documentación de Google Cloud [2].

3. Análisis con los conceptos

3.1. Transparencias de distribución

3.1.1. Acceso y ubicación

Spanner permite que una aplicación consulte y modifique la base de datos mediante bibliotecas cliente o una API, sin necesidad de conectarse por separado a cada réplica de la información. Además, las bibliotecas se encargan de aspectos como la gestión de sesiones y la ejecución de transacciones. Esto ofrece transparencia de acceso, ya que la aplicación no tiene que preocuparse por dónde se encuentran los datos; aun así, sigue siendo responsabilidad de la aplicación elegir la operación que desea realizar y la interfaz que va a utilizar [2, 4].

Asimismo, la ubicación física de cada fila queda en gran medida fuera de la vista de la aplicación: Spanner organiza los datos en rangos denominados *splits* y mantiene réplicas de cada uno. No obstante, esa transparencia no es absoluta. Al configurar la instancia se elige una configuración regional o multirregional, y la ubicación del líder puede influir en el tiempo que tarda una escritura. Por eso, aunque la aplicación no necesite localizar una fila para utilizarla, la ubicación sigue siendo un factor importante para el diseño y el rendimiento [2].

3.1.2. Migración y reubicación

En lo que se refiere a la migración y la reubicación, Spanner puede dividir un *split* cuando su carga aumenta, redistribuir los rangos resultantes entre los recursos disponibles y volver a fusionarlos cuando la carga disminuye. Esto permite que la distribución interna cambie sin que el usuario tenga que mover

manualmente cada fila. Aun así, ese movimiento no es del todo invisible en sus efectos: Google advierte que, si la división y el reequilibrio no avanzan al mismo ritmo que la carga, pueden aumentar la latencia y aparecer tiempos de espera o transacciones abortadas. Por ello, la transparencia en materia de migración y reubicación puede considerarse solo parcial [5].

3.1.3. Replicación, concurrencia y fallos

En el ámbito de la replicación, la concurrencia y los fallos, Spanner mantiene varias réplicas de cada *split* y se apoya en una votación basada en Paxos para confirmar las escrituras. El cliente trabaja con la base de datos sin tener que administrar cada una de esas copias por separado; en ese sentido, la replicación puede considerarse transparente. No obstante, el tiempo necesario para comunicarse con las réplicas y reunir una mayoría puede percibirse como latencia [2].

En cuanto a la concurrencia, el nivel de aislamiento serializable es el predeterminado para las transacciones de Spanner. El sistema coordina operaciones simultáneas para preservar ese aislamiento, pero una transacción puede abortarse por conflictos y requerir un nuevo intento. Así, Spanner evita que la aplicación tenga que coordinar directamente a las réplicas, aunque no le oculta por completo los conflictos entre transacciones [1].

La transparencia de fallos también tiene un límite. En una configuración regional base hay tres réplicas con voto, cada una en una zona distinta; si una falla, las otras dos aún pueden formar la mayoría necesaria para escribir. En cambio, si no existe esa mayoría, una escritura no puede confirmarse. La replicación permite soportar ciertos fallos, pero no garantiza que cualquier operación siga disponible ante cualquier interrupción [2].

3.2. CAP y PACELC

3.2.1. Prioridades ante una partición de red

Tomemos como caso la configuración regional base de tres réplicas. Si una zona queda separada de las otras dos, las dos que siguen comunicadas pueden reunir la mayoría y confirmar escrituras. La réplica aislada no puede confirmar por sí sola una escritura nueva. En este escenario, Spanner conserva la regla de confirmación por mayoría incluso cuando eso impide escribir desde el lado que perdió el quórum. Esta es la razón concreta para decir que, ante esa partición, prioriza la consistencia de las escrituras frente a la disponibilidad de escritura en la parte aislada; no significa que todo Spanner deje de funcionar [2].

3.2.2. Latencia y consistencia sin partición

La decisión sigue teniendo un costo. Una escritura pasa por la réplica líder y necesita votos de una mayoría antes de confirmarse; por ello, la comunicación entre réplicas forma parte de su latencia. El artículo de Zhang y colaboradores señala que, en las transacciones geodistribuidas, la coordinación y el consenso pueden exigir varias rondas de comunicación entre regiones. El artículo estudia una propuesta llamada D2PC: sus resultados de rendimiento no deben presentarse como mediciones de Spanner [2, 6].

Para las lecturas, la elección es más específica: Spanner ofrece lecturas fuertes, que incluyen los efectos de las transacciones confirmadas antes de comenzar la lectura, y modalidades que permiten leer datos de un momento anterior. Una lectura con antigüedad permitida puede atenderse desde una réplica cercana que ya tenga los datos correspondientes, mientras que una lectura fuerte desde una réplica no líder puede necesitar comunicación con el líder. En términos de PACELC, esto muestra un compromiso entre obtener datos recientes y reducir ciertos viajes de red; no sería preciso asignar una única prioridad de latencia o consistencia a todas las operaciones [1, 2].

3.3. Modelos de comunicación

Desde la aplicación, el modelo es cliente--servicio: el programa envía operaciones a Spanner mediante sus bibliotecas o API y recibe resultados o errores. Las bibliotecas oficiales se construyen sobre gRPC, y Google también ofrece una API RPC y una API REST. Por eso podemos identificar RPC como un mecanismo documentado de comunicación con el servicio, sin suponer que cada comunicación interna entre réplicas usa exactamente la misma interfaz pública [4].

Dentro del sistema, la comunicación cumple otra función. Cada *split* tiene un conjunto de réplicas y una de ellas actúa como líder. Para una escritura, el líder la remite a las demás réplicas con voto y espera las respuestas necesarias para alcanzar mayoría. En términos del análisis, esta coordinación se parece a un modelo con líder, no a una red donde todos los participantes cumplen siempre el mismo papel. Zhang y colaboradores describen además a Spanner como un ejemplo de coordinación de transacciones mediante 2PC sobre Multi-Paxos [2, 6].

La confirmación de una escritura depende de recibir votos, por lo que hay una espera sincrónica en esa parte del proceso. Después, las réplicas que aún no registraron la escritura pueden completarlo en segundo plano. Esto permite hablar de pasos sincrónicos y trabajo posterior asíncrono, pero la documentación consultada no justifica describir todas las comunicaciones internas como «mensajería» ni especificar el tipo de socket utilizado [2].

3.4. Tolerancia a fallos

3.4.1. Fallos soportados, detección y recuperación

Spanner combina copias de los datos, elección de líder y confirmación por mayoría. En la configuración regional de tres réplicas, puede seguir confirmando escrituras tras la pérdida de una réplica, porque permanecen dos votos. La documentación también indica que una réplica atrasada puede solicitar los datos que le faltan a otra para ponerse al día, y que el sistema de archivos subyacente ayuda a recuperar archivos tras fallos de máquina o disco. Estas son capacidades documentadas; no equivalen a afirmar que el servicio resiste cualquier número o tipo de fallos [2].

Respecto de la detección, la documentación de Spanner consultada permite afirmar que sus conjuntos de réplicas eligen un líder y que las bibliotecas cliente pueden gestionar reintentos ante errores transitorios. No aporta, en estas fuentes, un plazo universal ni un mecanismo específico de latidos para detectar todos los fallos; por ello, no conviene inventar esos detalles. El trabajo sobre HoliPaxos analiza detectores de fallos y lentitud como parte de otro protocolo de replicación: ayuda a entender el problema general, pero no prueba que Spanner implemente los detectores propuestos allí [2, 4, 7].

A nivel de cliente, los reintentos ofrecen otra vía de recuperación ante ciertos errores temporales. Google muestra, por ejemplo, cómo configurar una política para volver a intentar una operación que recibe `UNAVAILABLE`, con esperas entre intentos. Esos reintentos deben distinguirse de la recuperación de una réplica: actúan sobre la solicitud de la aplicación y no sustituyen el quórum interno [2, 8].

3.4.2. Incidente real documentado, si existe

El 4 de julio de 2023, Google informó que un número limitado de usuarios de Spanner tuvo errores al crear sesiones y utilizar API de control. El incidente duró aproximadamente dos horas y dieciséis minutos; Google señaló de forma preliminar una configuración incorrecta en el manejo de metadatos que sobrecargó esas operaciones. Algunos clientes con sesiones ya abiertas pudieron seguir trabajando, mientras que otros encontraron errores al reconectarse. El caso muestra un límite práctico de la transparencia de fallos en el acceso al servicio, pero no demuestra un fallo del consenso entre réplicas, porque el reporte no atribuye el incidente a ese mecanismo [9].

4. Experimentación propia

4.1. Montaje y procedimiento

El 13 de noviembre de 2026 se utilizó el emulador oficial de Spanner en un contenedor Docker local y la biblioteca de Spanner para Python. El contenedor expuso el servicio gRPC en `localhost:9010`; una variable de entorno dirigió el programa cliente a ese servicio. Google indica que el emulador conserva los datos en memoria y ofrece las API necesarias para desarrollar y probar aplicaciones, aunque no reproduce todas las características del servicio de producción [10].

La prueba creó una instancia, una base de datos y una tabla `Cuentas` con dos filas: la cuenta de Bryan comenzó con 100 unidades y la cuenta de destino con 50. Después de consultar ambos saldos, se preparó una transferencia de 10 unidades en una transacción que se canceló deliberadamente antes del commit. Por último, se repitió la transferencia y se permitió su confirmación. Esta secuencia se ejecutó mediante los pasos `preparar`, `consultar`, `cancelar` y `confirmar` del programa, en ese orden.

Como segunda prueba, se consultaron los saldos confirmados, se detuvo el contenedor y se repitió la consulta con un límite de 15 segundos. Después se inició el mismo contenedor, se consultó la instancia anterior y, finalmente, se volvieron a crear los datos de prueba.

4.2. Capturas y resultados

En la prueba de transferencia, los saldos iniciales de 100 y 50 se mantuvieron tras cancelar la transacción. Al confirmarla, pasaron a 90 y 60, con un total de 150.

4.2.1. Interrupción y recuperación del emulador

Con el emulador activo, la consulta de referencia devolvió los saldos de 90 y 60. Tras detener el contenedor, el comando de consulta no terminó dentro de los 15 segundos y el programa `timeout` salió con código 124.

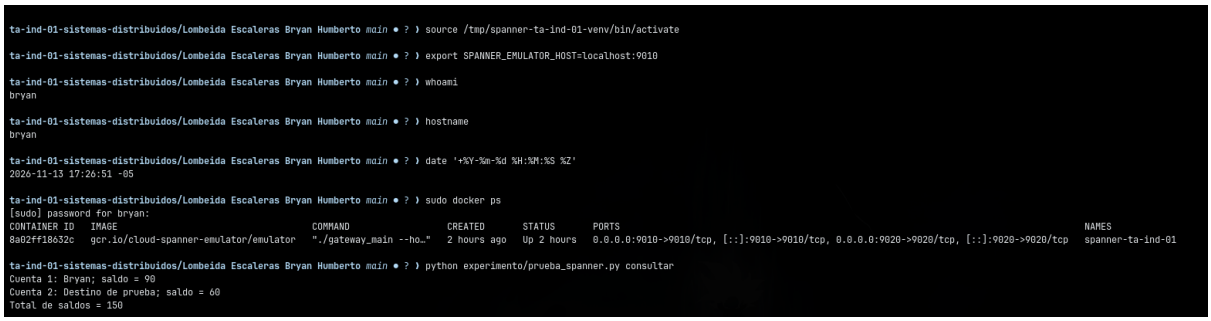


Figura 2: Consulta de referencia con el contenedor activo y los saldos de 90 y 60.

```
ta-ind-01-sistemas-distribuidos/Lombeida Escaleras Bryan Humberto main ● ? ❱ sudo docker stop spanner-ta-ind-01
spanner-ta-ind-01

ta-ind-01-sistemas-distribuidos/Lombeida Escaleras Bryan Humberto main ● ? ❱ sudo docker ps -a
CONTAINER ID   IMAGE                                     COMMAND                  CREATED       STATUS                     PORTS     NAMES
8a02ff18632c   gcr.io/cloud-spanner-emulator/emulator   "./gateway_main --ho…"   2 hours ago   Exited (2) 6 seconds ago             spanner-ta-ind-01

ta-ind-01-sistemas-distribuidos/Lombeida Escaleras Bryan Humberto main ● ? ❱ date '+%Y-%m-%d %H:%M:%S %Z'
2026-11-13 17:28:15 -05

ta-ind-01-sistemas-distribuidos/Lombeida Escaleras Bryan Humberto main ● ? ❱ timeout 15s python experimento/prueba_spanner.py consultar

ta-ind-01-sistemas-distribuidos/Lombeida Escaleras Bryan Humberto main ● ? × printf 'Código de salida: %s\n' "$?"
Código de salida: 124

ta-ind-01-sistemas-distribuidos/Lombeida Escaleras Bryan Humberto main ● ? ❱
```

Figura 3: Contenedor detenido y consulta interrumpida al alcanzar el límite de 15 segundos.

Tras el reinicio, Docker mostró el contenedor activo y el servicio respondió con `Instance not found`. La consulta volvió a funcionar después de ejecutar `preparar`, que creó la instancia y cargó los saldos de 100 y 50.

```
ta-ind-01-sistemas-distribuidos/Lombeida Escaleras Bryan Humberto main ● ? ❱ sudo docker start spanner-ta-ind-01
spanner-ta-ind-01

ta-ind-01-sistemas-distribuidos/Lombeida Escaleras Bryan Humberto main ● ? ❱ sudo docker ps
CONTAINER ID   IMAGE                                     COMMAND                  CREATED       STATUS         PORTS                                                
```

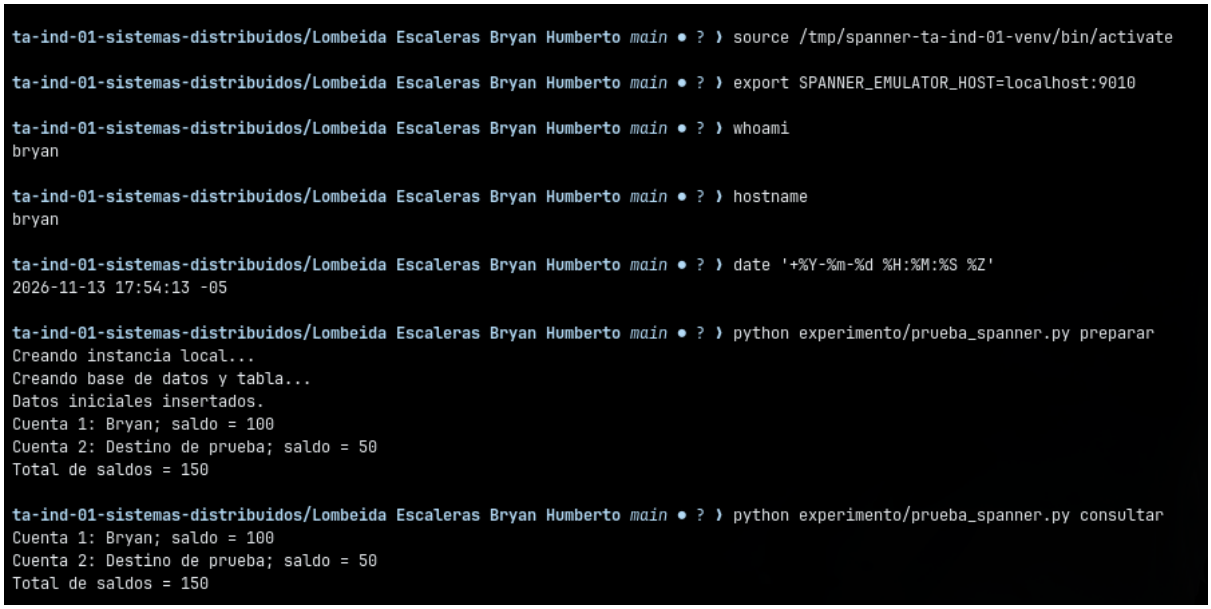


Figura 5: Creación de una instancia nueva y consulta de los saldos iniciales de 100 y 50.

4.3. Interpretación

La misma consulta permitió distinguir dos estados desde el cliente: al detener el contenedor, no hubo respuesta antes del límite fijado. Tras el reinicio, el servicio respondió, pero indicó que la instancia anterior ya no existía. El fallo fue visible para la aplicación y el acceso a los datos requirió recrear la instancia y cargar los registros iniciales. Ese comportamiento concuerda con el almacenamiento en memoria del emulador [10].

El experimento ilustra la comunicación cliente--servidor y el efecto observable de una interrupción local. Su alcance se limita a la disponibilidad de la consulta en este contenedor y a la recreación de sus datos. El código 124 registra el límite impuesto al comando, mientras que los mecanismos de replicación y quórum de Spanner se analizan con la documentación de producción [2, 10].

5. Ejemplos del entorno del estudiante

SGA de la UTEQ. Durante la matrícula intenté acceder al módulo dentro de la franja asignada a mi semestre. La página tardaba mucho en cargar y el módulo siguió lento ese día, aunque logré completar la matrícula. Este caso ilustra la latencia percibida: pude realizar la operación, pero tuve que esperar más de lo habitual.

WhatsApp. En ocasiones, al enviar varias imágenes, algunos archivos quedaron pendientes durante mucho tiempo y luego la aplicación mostró un error de envío. También me ocurrió con mensajes que siguieron sin enviarse después de reintentarlo. El caso ilustra la visibilidad de un fallo en la comunicación: la aplicación me avisó que el envío no se había completado y pude volver a intentarlo, aunque el reintento no siempre funcionó.

6. Análisis comparativo

La comparación se centra en Spanner con una configuración multirregional y en las tablas globales de DynamoDB con consistencia eventual multirregional (MREC). Se toma MREC como caso principal porque AWS lo aplica por defecto al crear una tabla global sin indicar un modo. También se considera la

opción de consistencia fuerte multirregional (MRSC) al discutir las decisiones de diseño [2, 11].

6.1. Tabla de comparación

Tabla 1: Spanner multirregional y DynamoDB Global Tables en modo MREC

Criterio	Google Spanner	DynamoDB (MREC)
Acceso y ubicación	El cliente usa la API de la base. El servicio dirige las escrituras al líder de las réplicas correspondientes [2, 4].	El cliente opera con elementos identificados por clave. El servicio localiza la partición en la región que atiende la solicitud [11, 12].
Comunicación	El cliente envía solicitudes al servicio. El líder comunica cada escritura a las réplicas con voto [2, 4].	El cliente envía solicitudes a la API. Un servicio de enrutamiento las dirige a los nodos que almacenan los datos [12].
Replicación	La escritura se confirma con el acuerdo de la mayoría de las réplicas con voto [2].	La réplica que recibe una escritura propaga el cambio a otras regiones de forma asincrónica [11].
Partición de red (CAP)	Sin una mayoría de votos, el grupo de réplicas afectado no puede confirmar escrituras [2].	Una réplica que sigue operativa puede aceptar escrituras locales aunque se interrumpa la replicación entre regiones [11].
Latencia y consistencia (PACELC)	La escritura espera el acuerdo del quórum. Una lectura que admita datos anteriores puede servirse desde una réplica cercana con la versión solicitada [2].	La escritura en una región no espera a que el cambio llegue a las demás. Una lectura en otra región puede devolver una versión anterior [11].
Concurrencia y transacciones	Los cambios de una transacción se aplican como unidad. El aislamiento serializable se usa por defecto [1].	La transacción es atómica en la región donde se ejecuta. Una réplica remota puede mostrar solo parte de los cambios durante la propagación [11, 13].
Recuperación ante fallos	Las escrituras pueden continuar si hay quórum. Una réplica atrasada puede solicitar a otra los cambios pendientes [2].	La aplicación puede redirigir el tráfico hacia otra réplica. Cuando se recupera una réplica aislada, se propagan los cambios pendientes [11].

6.2. Discusión de resultados

Los ejemplos de la sección 5 sirven para plantear dos preguntas separadas al comparar los sistemas: cuánto espera el usuario y si la operación logra completarse. La tabla 1 aplica esa distinción al tiempo necesario para confirmar una escritura y a la posibilidad de hacerlo cuando se interrumpe la comunicación entre réplicas [2, 11].

Para una transferencia como la del experimento, importa cuándo una consulta puede ver juntos el débito y el crédito. En Spanner, una transacción de lectura y escritura con el aislamiento predeterminado aplica ambos cambios como una sola operación. En DynamoDB con MREC, esa garantía se cumple en la región que procesa la transacción. Mientras se propagan los cambios, una lectura en otra región puede mostrar el débito de una cuenta sin reflejar todavía el crédito en la otra. Si ver ambos cambios juntos es un requisito de la aplicación, esa diferencia pesa en la elección del sistema [1, 11].

Para datos que se consultan por clave y pueden admitir un retraso entre regiones, la propagación asincrónica de MREC permite confirmar la escritura sin esperar a las réplicas remotas. Si dos regiones modifican el mismo elemento, se conserva la versión con la marca de tiempo interna más reciente. AWS también ofrece MRSC, que replica la escritura de forma sincrónica en al menos otra región antes de confirmarla y permite lecturas fuertes en cualquier réplica. La documentación de AWS indica que las tablas en modo

MRSC no admiten operaciones transaccionales [11, 12].

Lima y colaboradores examinaron otra decisión que afecta a la aplicación: dónde y cuándo leer un objeto que depende de cambios anteriores. Evaluaron sus estrategias de lectura con DynamoDB, Amazon S3 y MongoDB. En ese estudio, consultar más de una réplica podía reducir la espera, aunque también podía generar más solicitudes de lectura. El resultado ayuda a explicar por qué la estrategia del cliente importa cuando los datos se distribuyen entre regiones. Sus resultados corresponden a esas estrategias de acceso causal [14].

7. Postura crítica

7.1. Fortalezas

La principal fortaleza de Spanner es que permite mantener juntas las operaciones que forman parte de una misma decisión. Con el aislamiento serializable predeterminado, una transacción de lectura y escritura aplica todos sus cambios como una unidad. Además, si una transacción termina antes de que comience otra, el orden en que se observan sus efectos respeta esa secuencia. Esto resulta valioso al reservar la última unidad de un producto, porque la comprobación del inventario y el registro de la reserva pueden formar parte de la misma transacción [1].

La replicación también aporta continuidad frente a fallos parciales. Spanner puede seguir confirmando escrituras mientras conserve una mayoría de réplicas con voto. Además, cuando una aplicación acepta datos de un instante anterior, una réplica cercana que ya tenga esa versión puede atender la lectura. Esto permite reservar las lecturas fuertes para las operaciones que necesitan conocer el estado más reciente [2].

7.2. Limitaciones y compromisos de diseño

Confirmar una escritura exige que el líder obtenga el acuerdo de una mayoría. La latencia de esa coordinación puede aumentar cuando las réplicas con voto están en regiones alejadas. Si se pierde el quórum, las escrituras afectadas no se pueden confirmar. Por eso, al elegir dónde ubicar las réplicas conviene evaluar tanto el tiempo de respuesta habitual como los fallos que la aplicación debe soportar [2].

El aislamiento predeterminado también tiene un costo cuando muchas transacciones intentan modificar los mismos datos. Con el control de concurrencia pesimista, una transacción larga puede retener bloqueos y provocar esperas o abortos. Los reintentos ayudan a resolver conflictos transitorios, pero pueden aumentar el tiempo que tarda la aplicación en completar la operación. Estos efectos corresponden al servicio de producción y deben evaluarse allí. El emulador utilizado en el experimento bloquea toda la base durante una transacción de lectura y escritura y tiene un comportamiento de rendimiento diferente [1, 10].

7.3. Contextos en los que no conviene utilizarlo

Para un catálogo distribuido que consulta artículos por clave, actualiza cada elemento por separado y admite que otra región muestre temporalmente una versión anterior, DynamoDB con MREC puede encajar mejor. En ese modo, la confirmación de una escritura no espera la propagación a las demás regiones. La aplicación debe aceptar el retraso y considerar que, si dos regiones modifican el mismo elemento, prevalece la escritura con la marca de tiempo interna más reciente [11, 12].

Spanner tampoco sería la mejor opción si el requisito principal fuera seguir aceptando escrituras en una región aislada que ha perdido contacto con la mayoría. En ese escenario, el quórum impide confirmar esas operaciones. Si la aplicación puede aceptar divergencias temporales y resolverlas después, la replicación asincrónica de MREC ofrece una alternativa siempre que la réplica regional siga operativa [2, 11].

8. Conclusiones

En la transferencia de prueba, cancelar la transacción dejó los saldos en 100 y 50. Al confirmarla, los saldos cambiaron a 90 y 60 y el total se mantuvo en 150. Este resultado muestra por qué conviene agrupar el débito y el crédito en una sola transacción cuando la aplicación debe actualizar ambos saldos como una unidad [1].

La prueba de interrupción mostró que el estado del contenedor y la disponibilidad de los datos se deben verificar por separado. Mientras el contenedor estuvo detenido, la consulta quedó sin respuesta dentro del límite fijado de 15 segundos. Tras el reinicio, la instancia anterior ya no existía y la consulta volvió a funcionar después de crearla y cargar los saldos iniciales. El resultado corresponde al emulador, que guarda sus datos en memoria [10].

La comparación indica que Spanner es una opción adecuada si una consulta consistente debe poder observar juntos los cambios de una transacción y la aplicación puede asumir el tiempo que toma reunir el quórum de escritura. Para operaciones independientes que deben confirmarse en una región sin esperar la propagación y que admiten lecturas temporalmente desactualizadas en otras regiones, DynamoDB con consistencia eventual multirregional puede ajustarse mejor [1, 2, 11].

Declaración de uso de inteligencia artificial

Se utilizó OpenAI Codex como herramienta de apoyo, con el modelo GPT-6-Sol en la revisión final. Se empleó para organizar y revisar la redacción del informe, preparar el programa del experimento,como tambien guiar y explicar en el diagrama del boceto a mano.

Las afirmaciones técnicas donde se realizo una segunda revision con IA a base de la documentación oficial y los artículos citados. Los resultados del experimento se verificaron con las salidas de la terminal y las capturas originales. También se comprobó que el PDF compilara con las figuras y las citas.

Referencias

[1] Google Cloud. «Transactions overview,» visitado 12 de nov. de 2026. dirección: https://docs.cloud.google.com/spanner/docs/transactions

[2] Google Cloud. «Replication,» visitado 12 de nov. de 2026. dirección: https://docs.cloud.google.com/spanner/docs/replication

[3] S. Chatterjee y P. Mathur. «Spanner in 2025: Innovations powering intelligent, multi-model AI applications,» Google Cloud, visitado 13 de nov. de 2026. dirección: https://cloud.google.com/blog/products/databases/spanner-in-2025

[4] Google Cloud. «Overview of APIs, client libraries, and ORM drivers,» visitado 12 de nov. de 2026. dirección: https://docs.cloud.google.com/spanner/docs/api-libraries-overview

[5] Google Cloud. «Pre-splitting overview,» visitado 12 de nov. de 2026. dirección: https://docs.cloud.google.com/spanner/docs/pre-splitting-overview

[6] Z. Zhang, H. Hu, X. Zhou, Y. Tu, W. Qian y A. Zhou, «Fast Commitment for Geo-Distributed Transactions via Decentralized Co-coordinators,» *Proceedings of the VLDB Endowment*, vol. 17, n.º 10, págs. 2555-2567, 2024. DOI: 10.14778/3675034.3675046 dirección: https://www.vldb.org/pvldb/vol17/p2555-hu.pdf

[7] Z. Liang, V. Jabrayilov, A. Aghayev y A. Charapko, «HoliPaxos: Towards More Predictable Performance in State Machine Replication,» *Proceedings of the VLDB Endowment*, vol. 18, n.º 8,

págs. 2505-2518, 2025. DOI: 10.14778/3742728.3742744 dirección: https://www.vldb.org/pvldb/vol18/p2505-charapko.pdf

[8] Google Cloud. «Configure custom timeouts and retries,» visitado 12 de nov. de 2026. dirección: https://docs.cloud.google.com/spanner/docs/custom-timeout-and-retry

[9] Google Cloud Service Health. «Cloud Spanner unavailability or elevated errors: Incident report, 4 July 2023,» visitado 12 de nov. de 2026. dirección: https://status.cloud.google.com/incidents/Sen4ACpGsFmquzb8FxRj

[10] Google Cloud. «Emulate Spanner locally,» visitado 13 de nov. de 2026. dirección: https://docs.cloud.google.com/spanner/docs/emulator

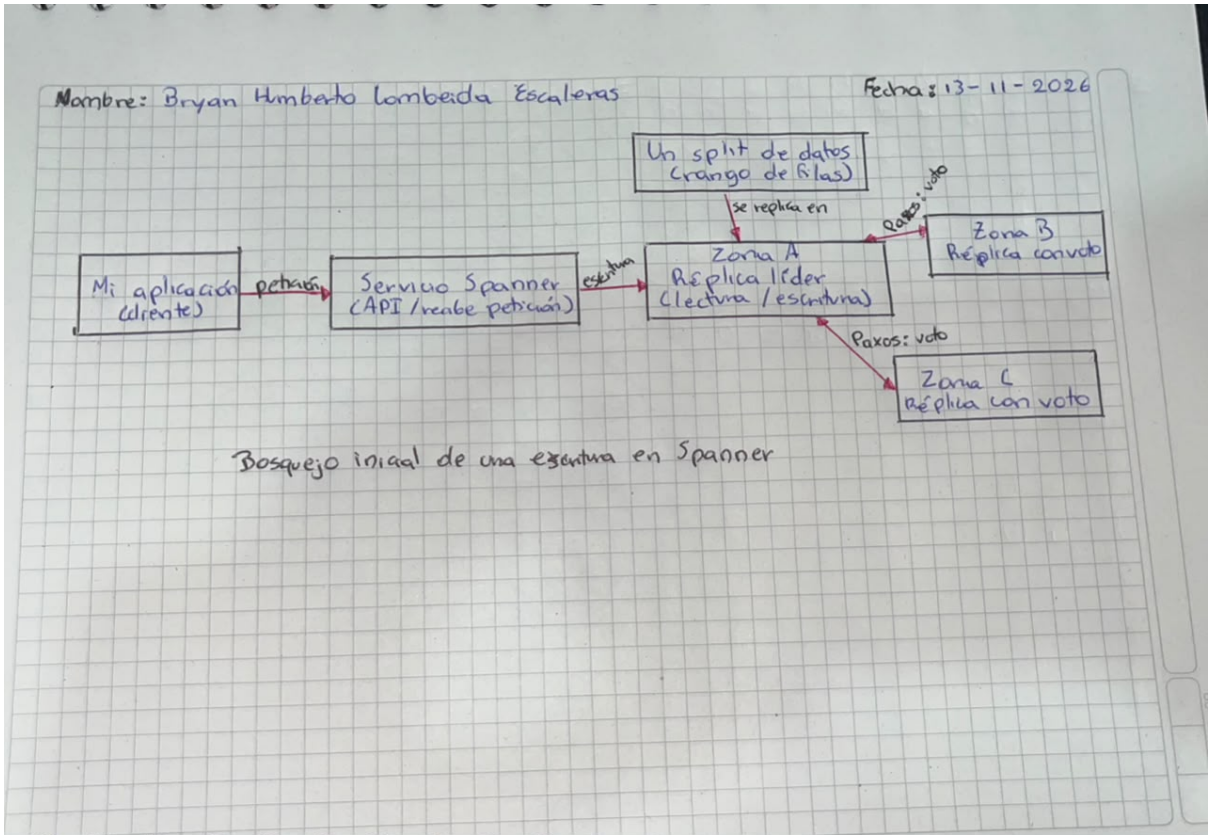
[11] Amazon Web Services. «How DynamoDB global tables work,» visitado 12 de nov. de 2026. dirección: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/V2globaltables_HowItWorks.html

[12] M. Elhemali et al., «Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service,» en *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, USENIX Association, 2022, págs. 1037-1048. dirección: https://www.usenix.org/system/files/atc22-elhemali.pdf

[13] J. Idziorek et al., «Distributed Transactions at Scale in Amazon DynamoDB,» en *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, USENIX Association, 2023, págs. 705-717. dirección: https://www.usenix.org/system/files/atc23-idziorek.pdf

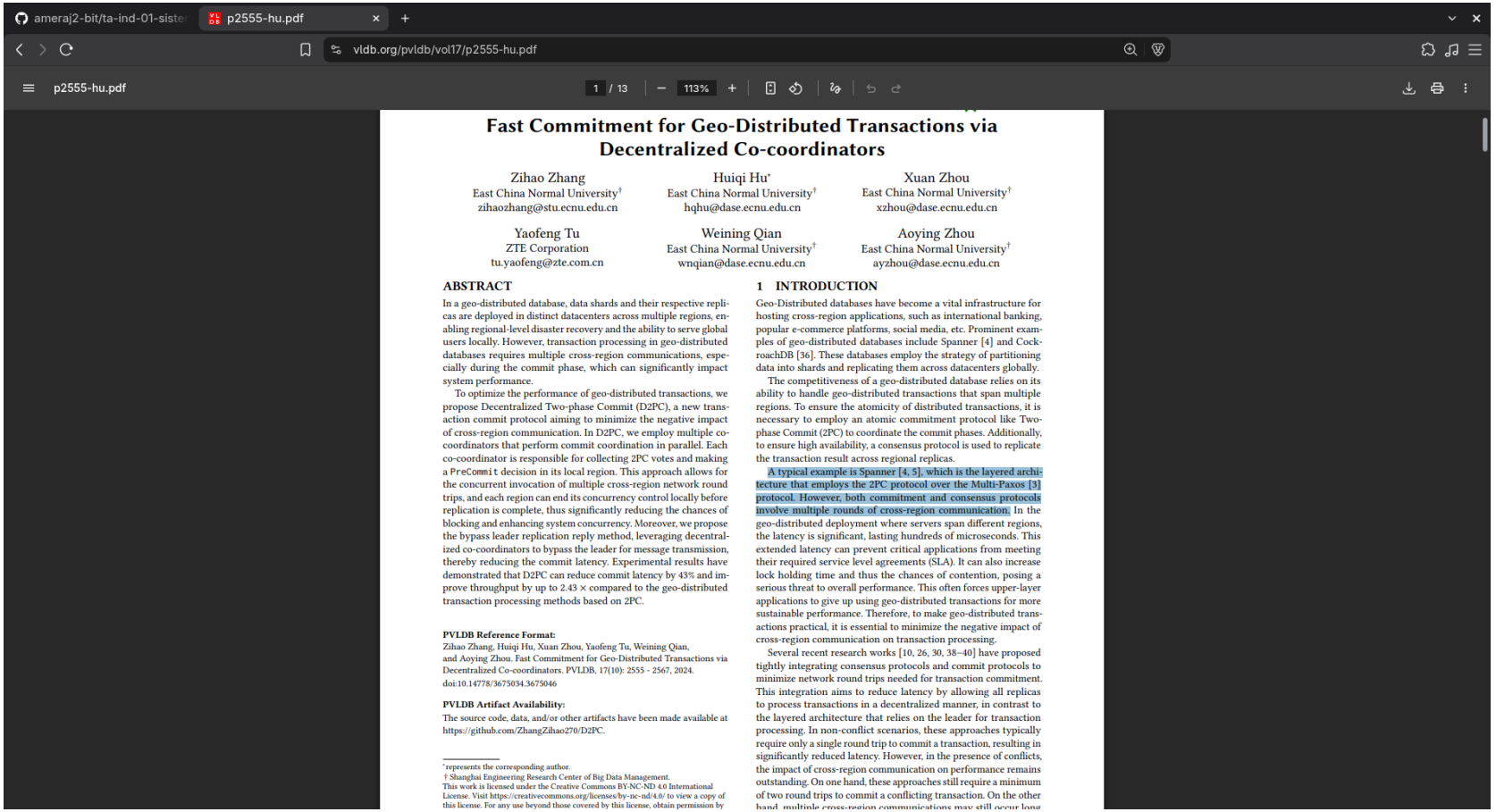
[14] S. Lima, F. Araujo, M. de Oliveira Guerreiro, J. Correia, A. Bento y R. Barbosa, «Efficient Causal Access in Geo-Replicated Storage Systems,» *Journal of Grid Computing*, vol. 21, 2023, art. 8. DOI: 10.1007/s10723-022-09640-z dirección: https://link.springer.com/article/10.1007/s10723-022-09640-z

Anexo A. Fotografía del esquema a mano

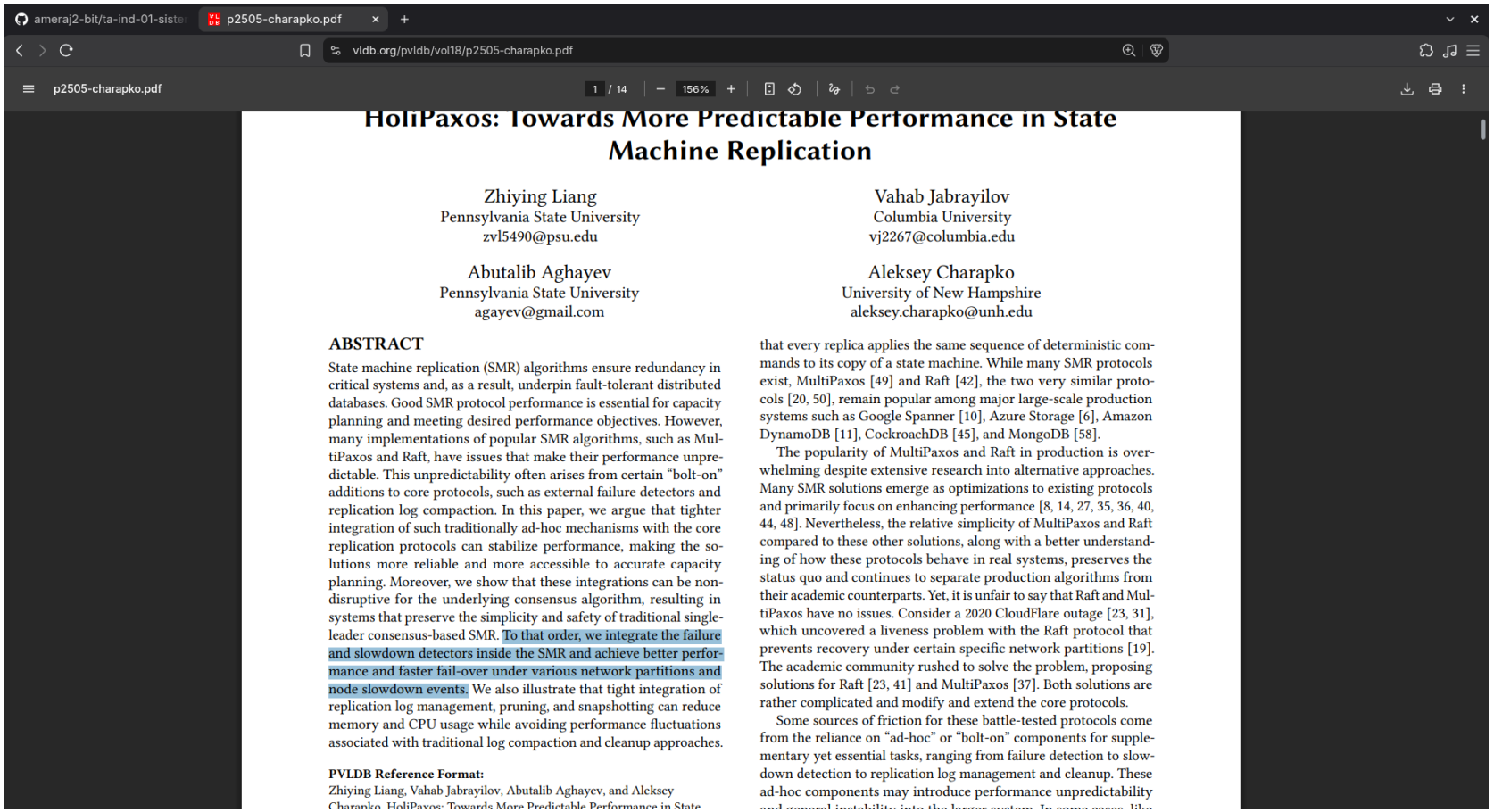


Bosquejo inicial de una escritura en Google Spanner, elaborado a mano el 13 de noviembre de 2026.

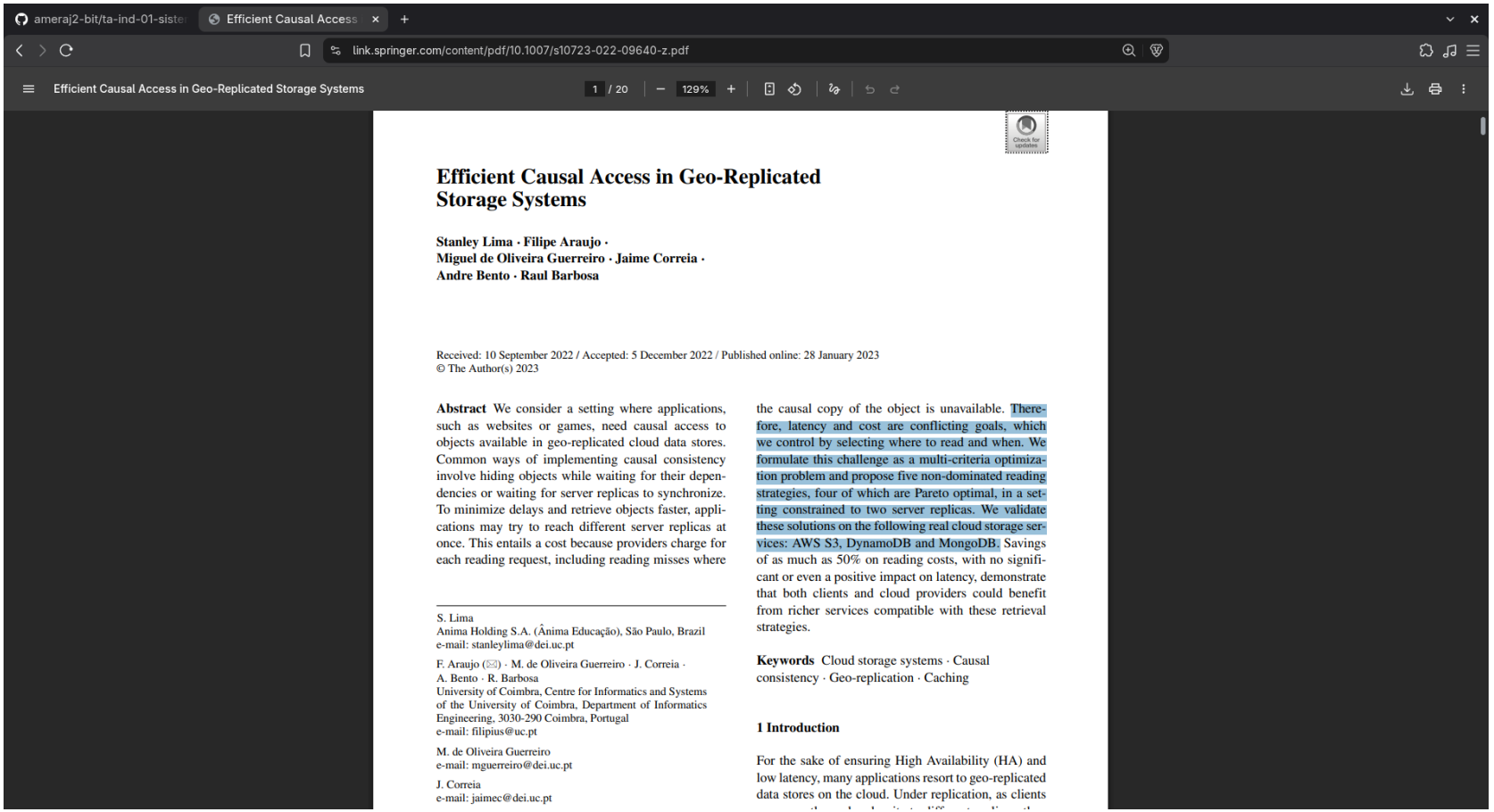
Anexo B. Capturas de lectura de las fuentes



Lectura anotada de Zhang y colaboradores, página 2555. El fragmento marcado describe la coordinación de Spanner mediante 2PC sobre Multi-Paxos y la comunicación entre regiones [6].



Lectura anotada de Liang y colaboradores, página 2505. El fragmento marcado trata la detección de fallos en HoliPaxos; se usa como apoyo conceptual, sin atribuir ese mecanismo a Spanner [7].



Lectura anotada de Lima y colaboradores, página 1 del PDF. El fragmento marcado presenta el equilibrio entre latencia y costo al decidir dónde y cuándo leer, y enumera los servicios usados para evaluar las estrategias, entre ellos DynamoDB [14].